

CSE446: Blockchain & Cryptocurrencies

Lecture - 5: Distributed Systems & Consensus Algorithms



Inspiring Excellence

Agenda

- Distributed system model (Node, process, network)
- Network types
- Various fault models
- Need for consensus
- Atomic broadcast, atomic broadcast properties
- Consensus properties
- FLP Impossibility, CAP Theorem
- Various consensus algorithms
- History and properties of money
- Money

Atomic broadcast properties

Properties	Note
Validity	This guarantees that if a message is broadcast by a valid node, it will be correctly included within the consensus protocol
Agreement	This is to guarantee that if a message is delivered to a valid node, it will ultimately be delivered to all valid nodes
Integrity	This is to ensure that a message is broadcast only once by a valid node
Total Order	This is to ensure that all nodes agree to the order of all delivered messages

Consensus protocol properties

Properties	Note
Safety/Consistency	A consensus protocol is considered safe (or consistent) only when all nodes produce the same valid output, according to the protocol rules, for the same atomic broadcast
Liveness/ availability	If all non-faulty participating nodes produce an output the protocol is considered live
Fault Tolerance	It exhibits the network's capability to perform as intended in the midst of node failures

FLP Impossibility

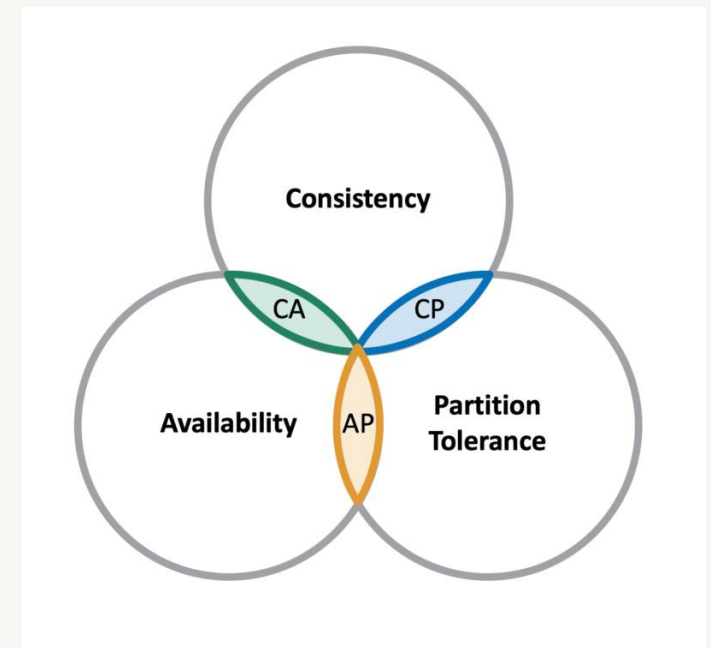
- A well-known theorem*, by Fischer, Lynch and Paterson, called FLP Impossibility has shown that
 - a deterministic consensus protocol cannot satisfy all three properties in an asynchronous network
- Safety and liveness are favoured over fault tolerance in the domain of distributed system applications

*M. J. Fischer, N. A. Lynch, and M. S. Paterson. "Impossibility of distributed consensus with one faulty process". Journal of the ACM (JACM), 32(2):374382, 1985

CAP Theorem

- CAP theorem* states that a shared replicated datastore (or, more generally, a replicated state machine) cannot achieve both consistency and availability when a network partitions in such a way that an arbitrary number of messages might be dropped

Pick any two



<https://hazelcast.com/wp-content/uploads/2021/12/cap-theorem-diagram-800x753-1.png>

*Brewer, E. A. "Towards robust distributed systems.". In PODC (Vol. 7), July 2000.

Crash-tolerant consensus algorithm

- Algorithms belonging to this class aim to guarantee the atomic broadcast (total order) of messages within the participating nodes in the presence of certain number of node failures
- These algorithms utilise the notion of views or epochs
 - which imply a certain duration of time
- A leader is selected for each epoch who takes decisions regarding the atomic broadcast, and all other nodes comply with its decision
- In case a leader fails due to a crash failure, the protocols elect a new leader to function

Crash-tolerant consensus algorithm

- The best-known algorithms belonging to this class can continue to function if the following condition holds:
 - $f < n/2$ where f is the number of faulty nodes and n is the total number of participating nodes
- Examples of some well-known crash-tolerant consensus protocol are
 - Paxos, Viewstamped Replication, ZooKeeper, and Raft

RAFT

- Raft is a distributed consensus algorithm
- It solves the problem of getting multiple servers to agree on a shared state even in the face of failures
- The shared status is usually a data structure supported by a replicated log
- We need the system to be fully operational as long as a majority of the servers are up

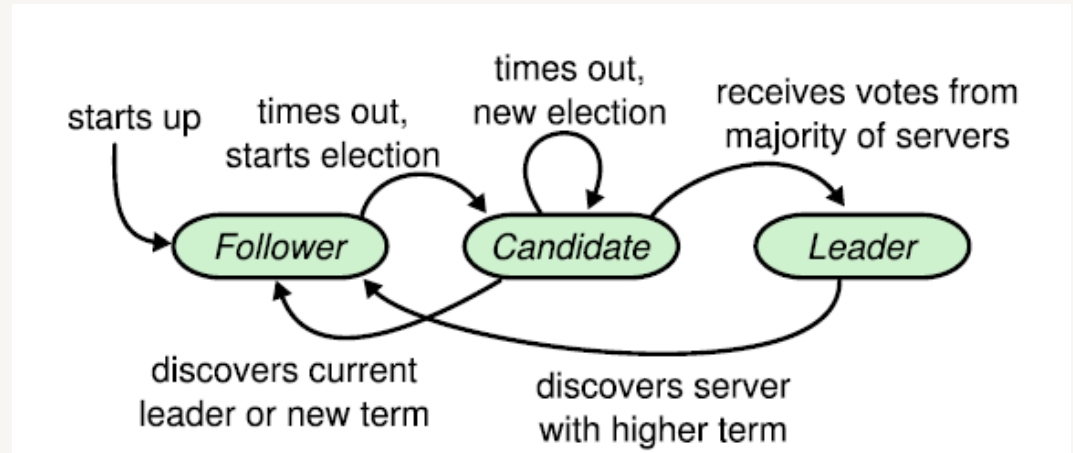
RAFT

- Raft works by electing a leader in the cluster
- The leader is responsible for accepting client requests and managing the replication of the log to other servers
- The data flows only in one direction: leader -> other servers
- Raft decomposes consensus into three sub-problems
 - Leader Election: A new leader needs to be elected in case of the failure of an existing one
 - Log replication: The leader needs to keep the logs of all servers in sync with its own through replication
 - Safety: to ensure consistency across all nodes

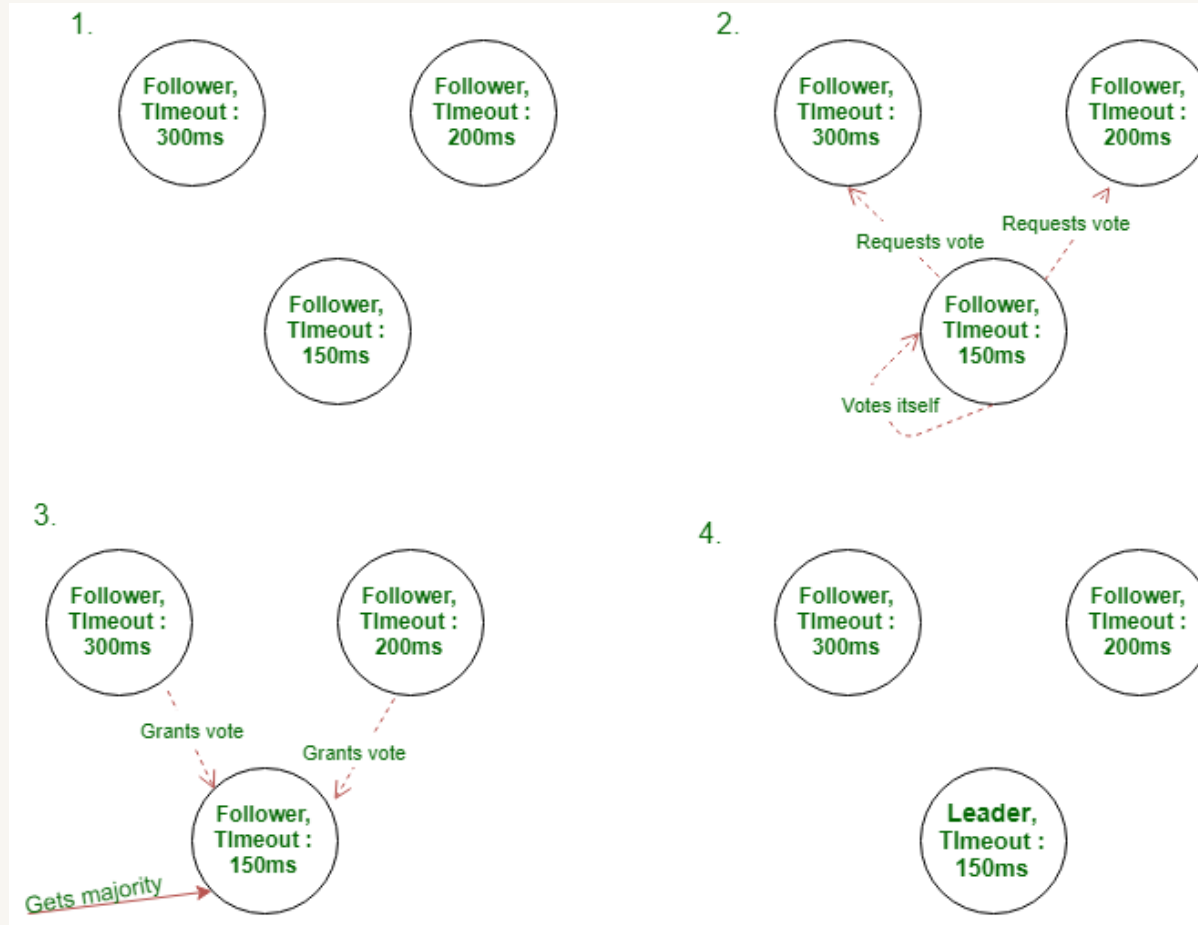
RAFT: leader election

- An epoch is defined as a term in RAFT
- A leader is selected in each term
- If a candidate wins the election, it remains the leader for the rest of the term
 - If the vote is split, then that term ends without a leader
- The term number increases monotonically and is also exchanged in every communication

Each node can only be a leader, follower, or candidate



RAFT: leader election



RAFT: log replication

- A client communicates with the leader to update the log (append to the log)
- The leader updates its log entry and replicates the value to the followers
- Each follower updates their log entries and replies back to the leader
- The leader commits the value to the log if the majority of the nodes reply back
- The leader replicates this commitment to the followers and they also commit the value

RAFT: safety

- RAFT ensures that it can provide consistency (safety) and availability when the network partitions using the following rule:
 - If one of the previous leaders has committed a log entry at a particular index, no other leader can apply a different log entry for that index
- This is assuming that this condition holds: $f < n/2$
- A fantastic visualisation of RAFT can be viewed from this location: <https://thesecretlivesofdata.com/raft>

Byzantine consensus algorithm

- Byzantine algorithms aim to reach consensus amidst of certain nodes exhibiting Byzantine behaviour
- Such Byzantine nodes are assumed to be under the control of an adversary and behave unpredictably with malicious intent
- Similar to any crash-tolerant consensus protocol, these protocols also utilise the concept of views/epochs
- A leader is elected in each view to order messages for atomic broadcast
 - other honest nodes are assumed to follow the instructions from the leader

Practical Byzantine Fault Tolerance (PBFT)

- One of the most well-known algorithms under this class is called Practical Byzantine Fault Tolerant (PBFT)
- It can achieve consensus in the presence of a certain number of Byzantine nodes under an eventual synchronous network assumption
- PBFT uses cryptographic algorithms such as signature, signature verification, and hash to ensure that everything stays irrevocable, unforgeable, and indisputable

Practical Byzantine Fault Tolerance (PBFT)

- The following conditions are required
- $f < n/3$, where f is the number of Byzantine nodes and n denotes the number of total nodes participating in the network, that is at least $n = 3f + 1$
 - A distributed system that contains $3f + 1$ nodes can have at most f faulty nodes (byzantine nodes)

PBFT: roles

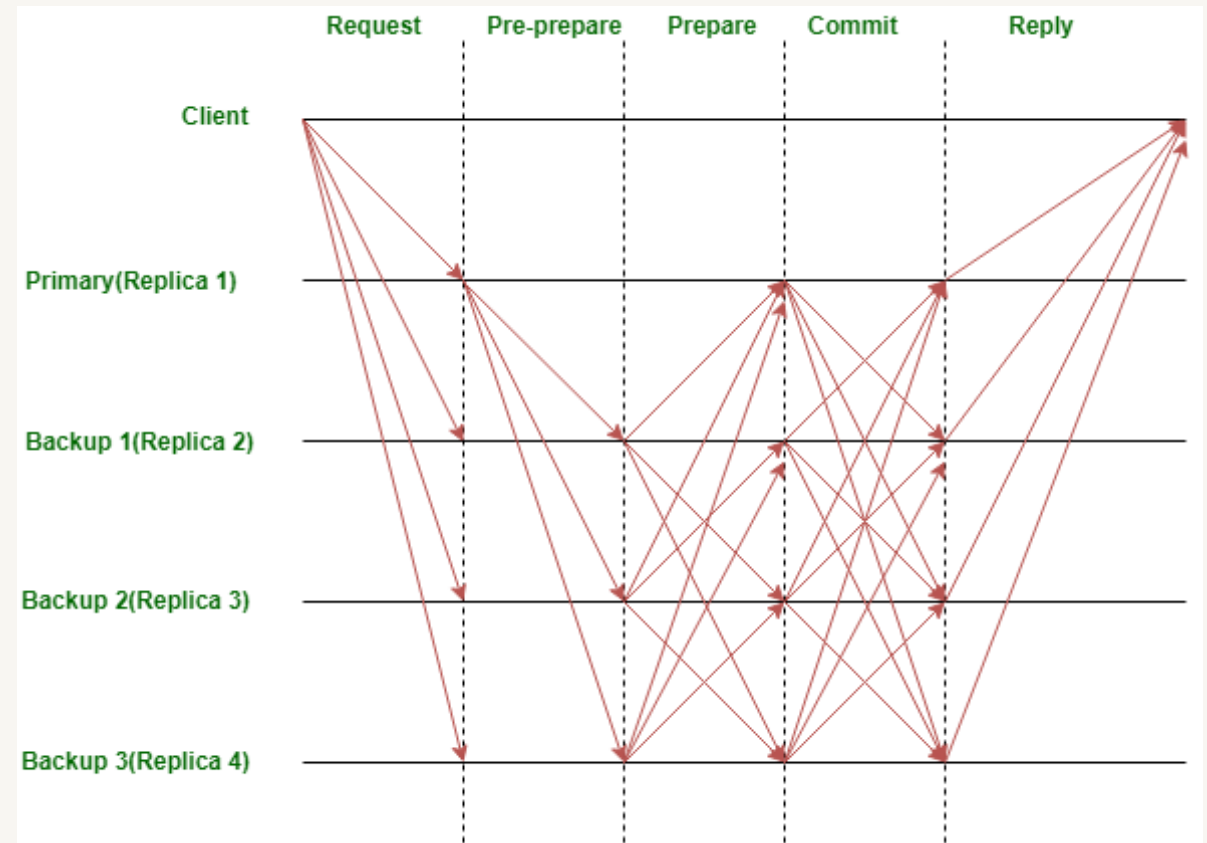
- An epoch is known as a view (a consensus round) in PBFT
- There are three different roles in PBFT
- Client:
 - Client nodes send transaction requests
- Primary:
 - These are main nodes which act as leaders
 - They initiate the consensus mechanism
 - In each view, there is one and only one primary node

PBFT: roles

- Replica:
 - These nodes are responsible for processing each request
 - There are many replica nodes and they all act in the same fashion
- Both Primary and Replica nodes are considered as consensus nodes, because they participate in the consensus process

PBFT: steps

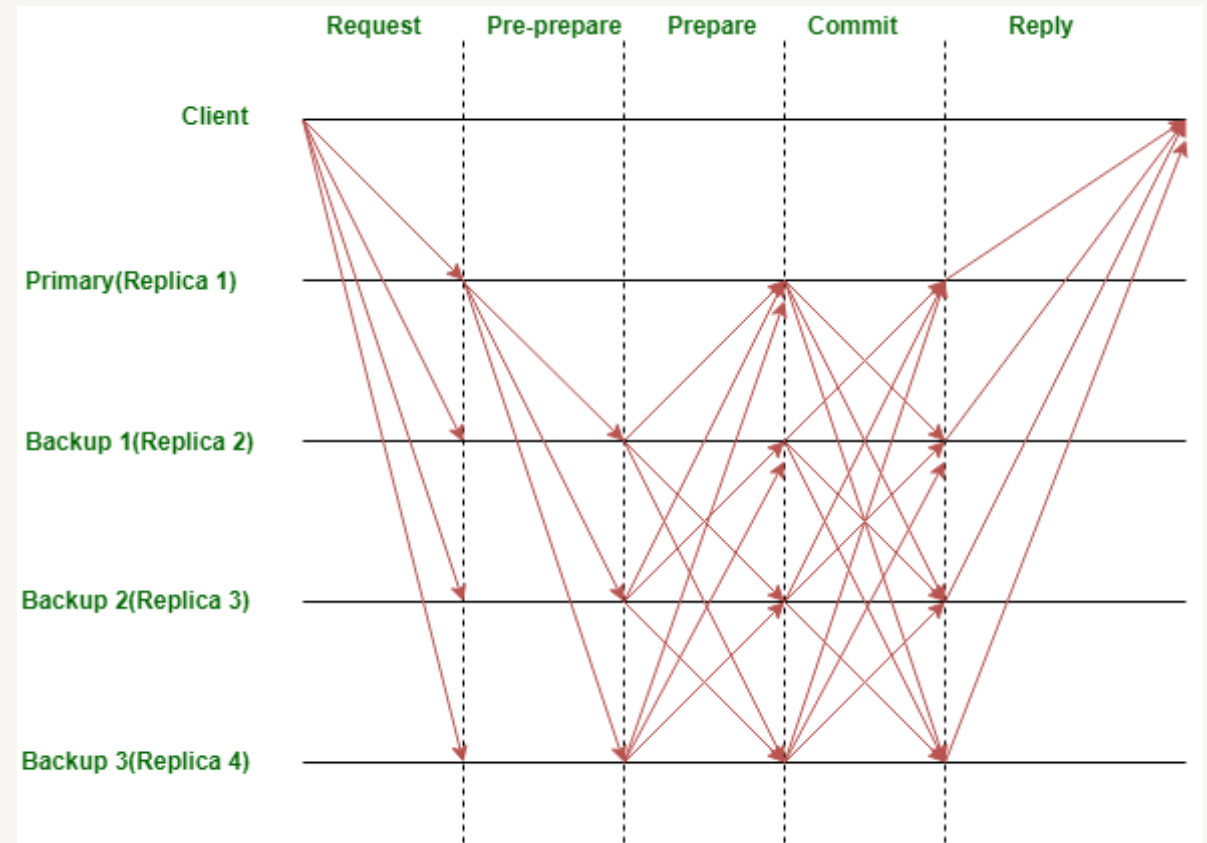
- Client sends a request to the system
- Consensus nodes (Replica1, Replica2, Replica3 & Replica4) receive the request
- Then the system participates in a three-phase consensus protocol: pre-prepare, prepare and commit



<https://media.geeksforgeeks.org/wp-content/uploads/pbft-algorithm-communication-visual.png>

PBFT: steps – pre~prepare

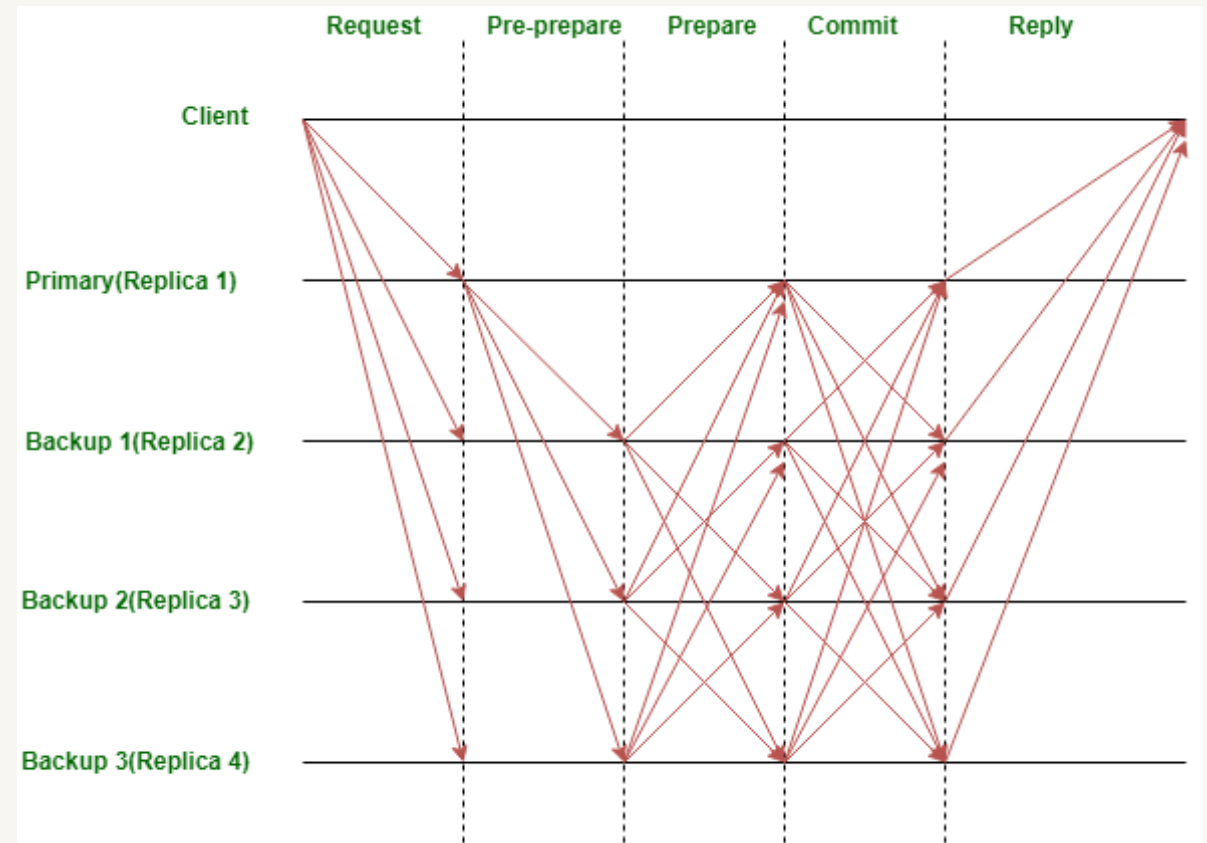
- Primary node verifies the request and generates a pre-prepare message which is broadcast to all replica nodes
- After receiving the messages, each replica node will validate the message
- If validated, each replica node will broadcast a corresponding prepare message



<https://media.geeksforgeeks.org/wp-content/uploads/pbft-algorithm-communication-visual.png>

PBFT: steps – prepare

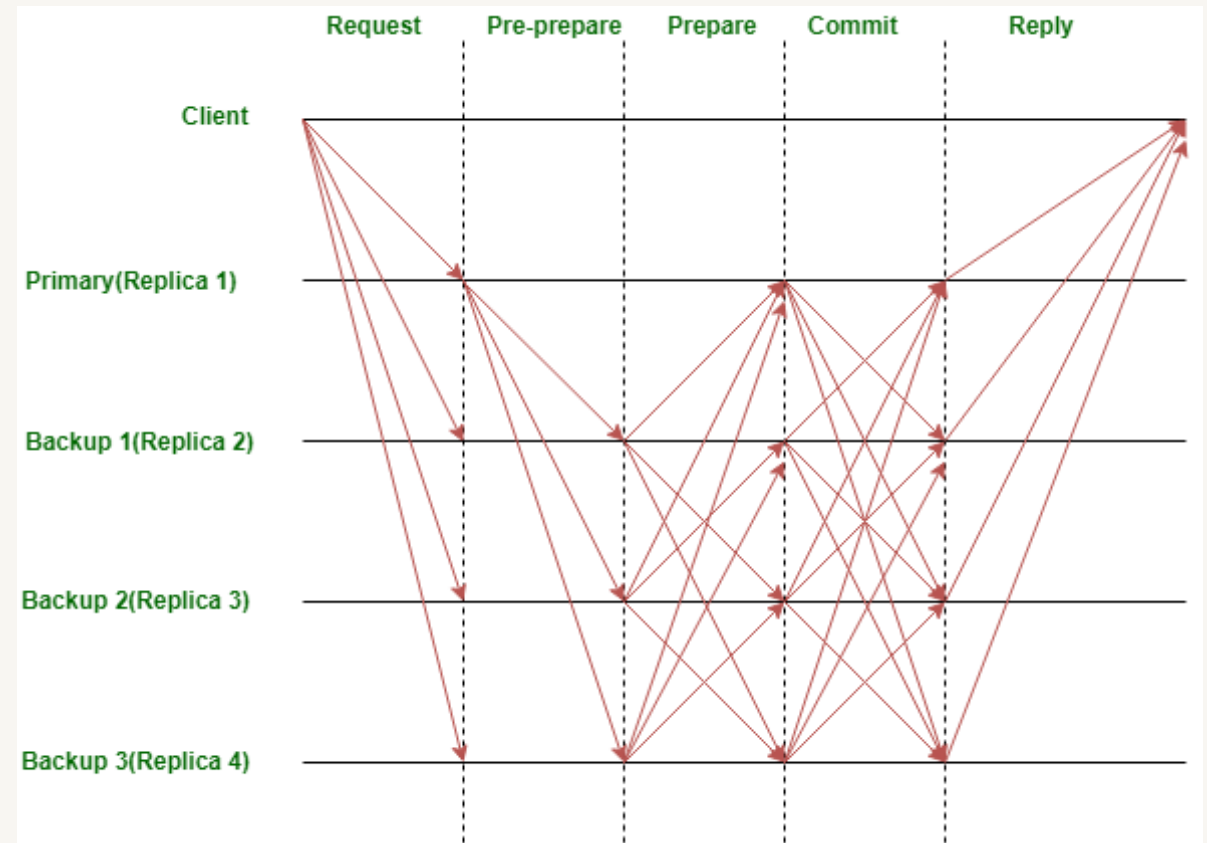
- Each node waits until $2f+1$ prepare messages are received
- Then each node will broadcast commit messages to other nodes
 - message implying that it is ready for processing the request



<https://media.geeksforgeeks.org/wp-content/uploads/pbft-algorithm-communication-visual.png>

PBFT: steps – commit

- Each node waits until $2f+1$ commit messages are received
- Then, the node processes each request, which changes the system states
- Each node replies back to the client
- The primary is rotated when there is a proposal from the replicas to do so or the primary remains unavailable for a duration
 - This mechanism is called a view change



<https://media.geeksforgeeks.org/wp-content/uploads/pbft-algorithm-communication-visual.png>

Let's talk about money....



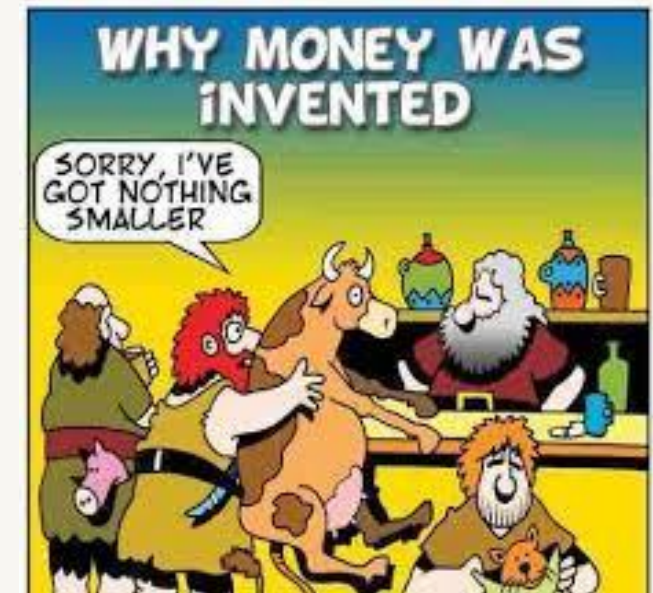
<https://media.tenor.com/Wkb4VIC1BHoAAAAC/money-money-rain.gif>



<https://media3.giphy.com/media/JpG2A9P3dPHXaTYrwu/giphy.gif?cid=ecf05e4785vmquns0oo0f7x59qjgnyjx467oxq37irfv5z7d&rid=giphy.gif&ct=g>

Money, money, money.....its history

- Money is a medium of exchange
- Stands in for an arbitrarily long chain of barter
 - exchanging goods or services for other goods or services without using a monetary unit
- Widely accepted



Money, money, money....its history

Bartering and Exchange

Bartering is exchanging products or services for commodities.



When one type of commodity is traded for another type of commodity, those commodities are called "**commodity money**," things that are used by most people, such as:



Taxes were first collected in Egypt close to 5,000 years ago when goods and labor were offered as ways of paying.



seeds



grain



other livestock



salt



tea



cattle

circa 1200 BC

Cowry Shells

Archaeologists believe that cowry shells were first used in **China**. These hardy, decorative shells have also been used as currency by other cultures, in other lands, at other times.



circa 1000 BC

Metals



The first use of metal as money is believed to have been around 3,000 years ago in **China**. The metal was formed into shapes to resemble **cowry shells**.



Sunny days have people tipping more than dreary days do.

TIP is the acronym for "To Insure Promptness."

First Coins

The first coins minted were probably the **Lydian electrum** tries from present-day **Turkey**. These coins, made of a gold and silver alloy, sported a lion's head on one side.



circa 600 BC

Money, money, money.....its history

Paper Money



Paper money first made its way into marketplaces a mere 1,000 years ago during the Tang dynasty in **China**. Numismatics, scientists who study the history of money, believe that this was the byproduct of **block printing**.



Marco Polo was amazed by the paper currency he saw during his travels through China.



Using the word "buck" to mean a dollar comes from the days when many Americans traded animal skins, including those from deer and elk bucks.

Banking in Europe



The modern form of banking had its birth in medieval and early **Renaissance Italy**, particularly in the nation's wealthy northern cities. One of the most famous of the banks from this period is the Medici bank founded by **Giovanni Medici** in 1397.



Wampum

The **Massachusetts Bay Colony** declared wampum, strings of clamshells used as money by American Indians, legal tender.

The paper used in U.S. bills is not made from trees, but rather made up of:



75% cotton



25% linen



The Gold Standard



Britain pegged its currency to gold to help govern inflation. **America** went on the gold standard in 1900. They have both since left the gold standard.

The U.S. government began to print paper "green-backs" to finance the Civil War in July of 1861. These first paper notes were worth 1 cent, 5 cents, 25 cents, and 50 cents.

circa
1000 AD

circa
1400 AD

1637

1816

Properties of money

- Money is a unit of account
 - a nominal monetary unit of measure used to represent the real value (or cost) of any economic item; i.e. goods, services, assets, liabilities, income, expenses
- Money is a store of value
- Recognisable
- Fungible (mutually interchangeable)



<https://upload.wikimedia.org/wikipedia/commons/e/e3/Gold-295936.jpg>

Properties of money

- Divisible
- Transportable
- Transferable
- Hard to counterfeit
- Stable supply



<https://upload.wikimedia.org/wikipedia/commons/e/e3/Gold-295936.jpg>

Money, money, money.....its history

- With the evolution of the Internet, the need for digital currency became clear

SSL / TLS – 1996

HTTP- 1990

TCP/IP - 1974

Ethernet - 1974



Money, money, money.....its history

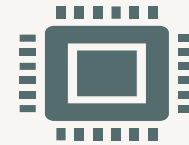


DigiCash – the first ever digital currency invented by David Chaum in 1983, commercialised in 1990, went bankruptcy in 1998 with the paper:

“Blind signatures for untraceable payments”

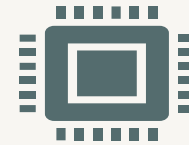


b-money, an anonymous, distributed electronic cash system – by Wei Dai in 1998



Bit Gold a proposal by Nick Szabo in 1998 where he conceptualised the idea of utilising cryptographic puzzles for unforgeable proof of work chains: a direct pre-cursor of Bitcoin idea

Money, money, money.....its history



All these ventures or ideas failed because they utilised mostly centralised components. And the puzzle to move money online between two entities seems like an elusive idea...until??

DigiCash
digital
invest
Ch
com
1990, went bankrupt
in 1998 with the paper:

"Blind signatures for
untraceable payments"

sal by
1998
the
ng
cryptographic puzzles
for unforgeable proof
of work chains: a direct
pre-cursor of Bitcoin
idea

